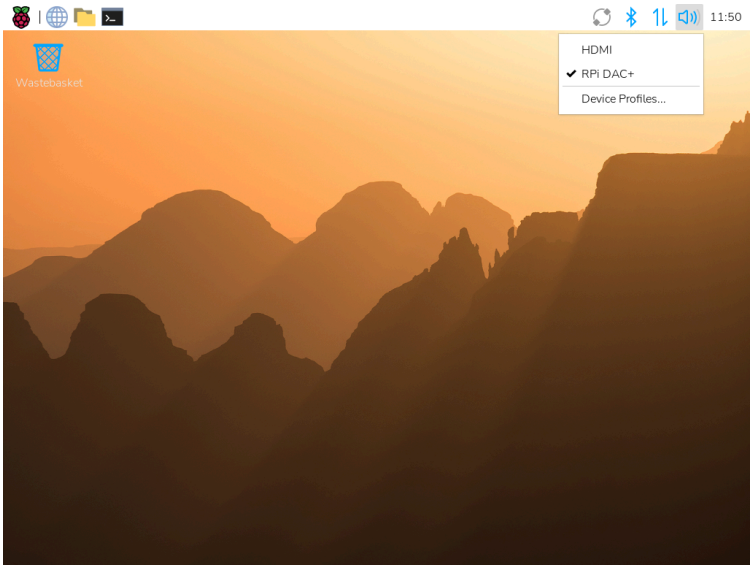
Raspberry Pi audio board configuration

Source: https://github.com/raspberrypi/documentation/blob/7376533ab86ea07acc77ef6a9f3c3e57fcff833d/documentation/asciidoc/accessories/audio/configuration.adoc (CC BY-SA 4.0; adapted for this benchmark)
Image source:
https://raw.githubusercontent.com/raspberrypi/documentation/7376533ab86ea07acc77ef6a9f3c3e57fcff833d/documentation/asciidoc/accessories/audio/images/gui.png (CC BY-SA 4.0)



Raspberry Pi audio board configuration

Attach, connect, and configure

Raspberry Pi audio boards are designed to be plug-and-play with Raspberry Pi devices running Raspberry Pi OS, as they feature a pre-programmed EEPROM. They belong to a family of devices known as HATs (Hardware Attached on Top).

They can also be used with a headless (remote) setup.

Attach and connect the audio board

Raspberry Pi audio boards attach to the 40-pin GPIO header and are designed to be supported using the supplied circuit board standoffs and screws. All necessary mounting hardware, including spacers, screws, and connectors, is provided.

Soldering isn't required for normal operation, but might be if you want to hard-wire specific connectors (External Line Return connections on the DAC Pro, for example).

To attach the audio board:

. Screw the PCB spacers finger-tight onto your Raspberry Pi device.

. Fit the audio board onto the 40-pin GPIO header.

. From above the board, screw in the remaining screws.

Check hardware version

There are multiple versions of Raspberry Pi audio boards. The version determines the actions required for configuration.

Older IQaudIO-branded boards have a black PCB. Newer Raspberry Pi-branded boards have a green PCB. These boards are electrically equivalent, but have different EEPROM contents.

To check the version of your audio board:

. Attach the audio board to your Raspberry Pi and apply power to it.

. Check that the power LED on the audio board is illuminated, if it has one. For example, the Codec Zero has an LED marked PWR.

. On the Raspberry Pi device, use the Terminal to enter the following command:

+

$ grep -a . /proc/device-tree/hat/

+

The Terminal returns one of the following strings:

Raspberry Pi Ltd.::

+

No further action is required (see below for <<extra-codec-zero-configuration>>).

IQaudIO Limited.::

+

No further action is required (see below for <<extra-codec-zero-configuration>>).

No such file or directory::

+

--

The HAT isn't detected. Do the following:

. Power down the device.

. Check the physical connections between the HAT and the Raspberry Pi device.

. Power on the device, then repeat Step 3 in the preceding section.

. If the same string is returned, open config.txt on your Raspberry Pi device.

. Add the following lines to cancel loading the default HAT overlay and apply the correct device tree overlay:

+

Suppress the default HAT overlay:

dtoverlay=

Add ONE of the following:

dtoverlay=rpi-codeczero

dtoverlay=rpi-dacplus

dtoverlay=rpi-dacpro

dtoverlay=rpi-digiampplus

. Save and close the file.

. Reboot your Raspberry Pi to make the changes take effect.

--

Configure the audio board

After you've attached and connected your audio board to your Raspberry Pi device, you can switch between using it and the on-board device for audio playback:

. Right-click the audio settings in the top-right corner of your screen.

. Switch between the on-board audio settings and the HAT audio settings.

You can also disable the on-board audio device, which makes the Raspberry Pi audio board the primary audio device in Raspberry Pi OS.

The command-line method described below is useful if you're running a headless setup (or Raspberry Pi OS Lite) and can't control the audio board through the graphical Raspberry Pi OS desktop interface.

To disable the on-board audio device using SSH:

. Connect to your Raspberry Pi device using SSH.

. Run the following command to edit the file:

+

$ sudo nano /boot/firmware/config.txt

+

. Find the dtparam=audio=on line in the file and comment it out by placing a # symbol at the start of the line.

+

The program disregards anything written after the # symbol on any given line. Your /boot/firmware/config.txt file now includes the following entry:

+

#dtparam=audio=on

. Press Ctrl+X, then Y, then Enter to save.

. Reboot your Raspberry Pi for the settings to take effect:

+

$ sudo reboot

Extra Codec Zero configuration

Raspberry Pi Codec Zero uses the Dialog Semiconductor DA7212 codec.

This allows the recording of audio from the built-in MEMS microphone, from stereo headers (AUX IN), or from two mono external electret microphones.

Playback is through stereo headers (AUX OUT) or a mono speaker connector.

Each input and output device has its own mixer, allowing audio levels and volume to be adjusted independently. Within the codec itself, other mixers and switches exist to allow the output to be mixed to a single mono channel for single-speaker output. Signals can also be inverted; there's a five-band equaliser to adjust certain frequency bands. You can control these settings interactively using AlsaMixer, or programmatically.

Both AUX IN and AUX OUT are 1 V RMS. You might need to adjust the mixer for AUX IN to ensure the input signal doesn't saturate the ADC. Similarly, adjust the output mixers to get the best possible output.

Preconfigured scripts (loadable ALSA settings) are available on GitHub, offering:

Mono MEMS microphone recording, mono speaker playback.

Mono MEMS microphone recording, mono AUX OUT playback.

Stereo AUX IN recording, stereo AUX OUT playback

Stereo MIC1/MIC2 recording, stereo AUX OUT playback

Codec Zero needs to know which of these input and output settings are used each time your Raspberry Pi powers on.

To provide the codec with input and output settings:

. If git isn't installed, open the Terminal and run the following command:

+

$ sudo apt install git

+

. In a terminal session, run the following command to clone the scripts:

+

$ git clone https://github.com/raspberrypi/Pi-Codec.git

+

. Run the following command to set your device to use the on-board MEMS microphone for input and speaker playback for output, replacing <username> with your username:

+

$ sudo alsactl restore -f /home/<username>/Pi-Codec/CodecZeroOnboardMICrecordandSPKplayback.state

+

This command might result in harmless warnings, including:

+

"failed to import hw" or "sndusecasemgropen"

"No state is present for card"

+

In most cases, you can safely ignore these warnings.

+

However, the following warnings can indicate a hardware failure:

+

"Remote I/O error"

+

. To make sure your settings persist the next time you boot the Raspberry Pi device, edit the /etc/rc.local file:

+

$ sudo nano /etc/rc.local

+

. Add the chosen script command above the exit 0 line.

+

The file looks similar to the following:

+

#!/bin/sh

#

rc.local

#

This script is executed at the end of each multiuser runlevel.

Make sure that the script will "exit 0" on success or any other

value on error.

#

In order to enable or disable this script just change the execution

bits.

#

By default this script does nothing.

sudo alsactl restore -f /home/<username>/Pi-Codec/CodecZeroOnboardMICrecordandSPKplayback.state

exit 0

+

. Press Ctrl+X, then Y, then Enter to save. Reboot for the settings to take effect:

+

$ sudo reboot

If you're using your Raspberry Pi in a headless or command-line environment:

. Create the following file in your home folder:

+

$ nano ~/.asoundrc

+

. Add the following lines to the file to route default playback and capture through the ALSA plug plugin for automatic format and channel conversions:

+

pcm.!default {

type plug

slave.pcm {

type hw

card Zero

}

}

ctl.!default {

type hw

card Zero

}

+

. Press Ctrl+X, then Y, then Enter to save.

. Reboot again to complete the configuration:

+

$ sudo reboot

Audio frameworks

Like other modern Linux distributions, Raspberry Pi OS uses PulseAudio or PipeWire by default for audio control.

These frameworks can mix and switch audio from multiple sources, and provide a high-level API for audio applications. Many audio applications use these frameworks by default.

Only create ~/.asoundrc if an audio application needs to:

Communicate directly with ALSA.

Run in an environment where PulseAudio or PipeWire isn't present.

This file can interfere with the UI view of underlying audio resources. We don't recommend creating ~/.asoundrc when running the Raspberry Pi OS desktop. The UI might automatically remove this file if it exists.

Mute and unmute the DigiAMP+

The mute state on DigiAMP+ is toggled using GPIO22. The device tree overlay supports unmuting DigiAMP+ using additional parameters.

To apply a one-shot unmute when the kernel module loads:

For Raspberry Pi boards:

dtoverlay=rpi-digiampplus,unmuteamp

For IQaudIO boards:

dtoverlay=iqaudio-digiampplus,unmuteamp

To unmute the amplifier when an ALSA device is opened by a client and mute with a five-second delay when closed (reopening within the five-second window cancels the mute):

For Raspberry Pi boards:

dtoverlay=rpi-digiampplus,automuteamp

For IQaudIO boards:

dtoverlay=iqaudio-digiampplus,automuteamp

If you don't want to control the mute state through the device tree, you can script your own solution.

The amplifier starts muted.

To unmute the amplifier:

$ pinctrl 22 op dh

To mute the amplifier again:

$ pinctrl 22 dl

Controlling the digital volume

If you're using the Raspberry Pi OS desktop environment, you can use the volume slider in the system tray.

To adjust the audio output volume on your audio board in a headless setup, use AlsaMixer:

$ alsamixer

Use the left and right arrow keys to select the volume channel you want to adjust, then use the up and down arrow keys to adjust the volume. Press Esc to exit AlsaMixer.